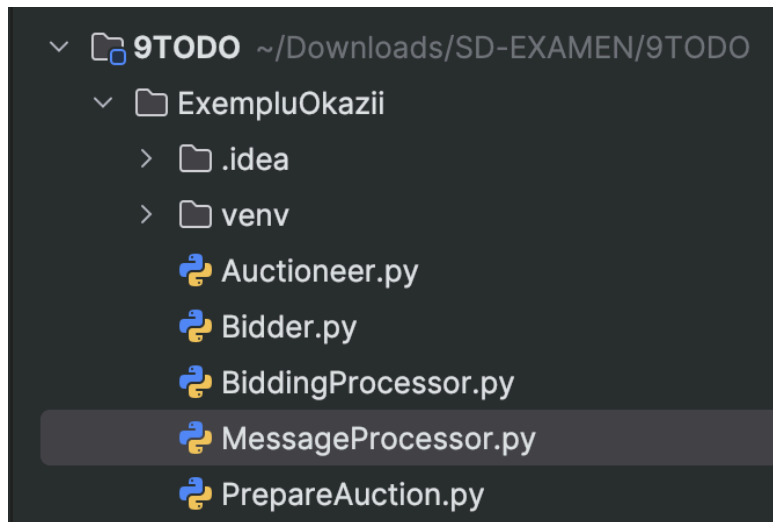


Pornind de la aplicația exemplu din laboratorul 10, să se modifice aceasta astfel încât să se introducă un procesor de flux care să monitorizeze toate operațiile efectuate de celelalte procesoare de flux în sensul scrierii într-un jurnal local. Se vor utiliza microservicii Python și o comunicare prin RabbitMQ.

Dacă se consideră necesar se mai pot crea oricâte servicii se dorește conform gândirii studentului. Se vor respecta principiile SOLID pentru microservicii. La prezentare se va arăta diagrama de clase și se va explica aceasta împreună cu gradul de respectare a principiilor. Apoi va urma discuția pe cod.



Notite

1. Vezi in Lab despre rularea proiectului si daca zice ceva de kafka relevant

Auctioneer.py

```
from kafka import KafkaConsumer, KafkaProducer

class Auctioneer:
    def __init__(self, bids_topic, notify_message_processor_topic):
        super().__init__()
        self.bids_topic = bids_topic
        self.notify_processor_topic = notify_message_processor_topic

        # consumatorul pentru ofertele de la licitatie
        self.bids_consumer = KafkaConsumer(
            self.bids_topic,
            auto_offset_reset="earliest", # mesajele se preiau de la cel mai
            # vechi la cel mai recent
```

```

        group_id="auctioneers",
        consumer_timeout_ms=15_000 # timeout de 15 secunde
    )

    # producatorul pentru notificarea procesorului de mesaje
    self.notify_processor_producer = KafkaProducer()

    def receive_bids(self):
        # se preiau toate ofertele din topicul bids_topic
        print("Astept oferte pentru licitatie...")
        for msg in self.bids_consumer:
            for header in msg.headers:
                if header[0] == "identity":
                    identity = str(header[1], encoding="utf-8")
                elif header[0] == "amount":
                    bid_amount = int.from_bytes(header[1], 'big')

            print("{} a licitat {}".format(identity, bid_amount))

        # bids_consumer genereaza exceptia StopIteration atunci cand se atinge
        # timeout-ul de 10 secunde
        # => licitatia se incheie dupa ce timp de 15 secunde nu s-a primit
        # nicio oferta
        self.finish_auction()

    def finish_auction(self):
        print("Licitatia s-a incheiat!")
        self.bids_consumer.close()

        # se notifica MessageProcessor ca poate incepe procesarea mesajelor
        auction_finished_message = bytearray("incheiat", encoding="utf-8")
        self.notify_processor_producer.send(topic=self.notify_processor_topic,
        value=auction_finished_message)
        self.notify_processor_producer.flush()
        self.notify_processor_producer.close()

    def run(self):
        self.receive_bids()

if __name__ == '__main__':
    auctioneer = Auctioneer(
        bids_topic="topic_oferte",
        notify_message_processor_topic="topic_notificare_procesor_mesaje"
    )
    auctioneer.run()

```

Bidder:

```

from kafka import KafkaProducer, KafkaConsumer
from random import randint
from uuid import uuid4

class Bidder:
    def __init__(self, bids_topic, result_topic):
        super().__init__()
        self.bids_topic = bids_topic
        self.result_topic = result_topic

        # producatorul pentru oferte de licitatie
        self.bid_producer = KafkaProducer()

        # consumatorul pentru rezultatul licitatiei
        self.result_consumer = KafkaConsumer(
            self.result_topic,
            auto_offset_reset="earliest" # mesajele se preiau de la cel mai
vechi la cel mai recent
        )

        self.my_bid = randint(1000, 10_000) # se genereaza oferta ca numar
aleator intre 1000 si 10.000
        self.my_id = uuid4() # se genereaza un identificator unic pentru
ofertant

    def bid(self):
        # se construiesc mesajul pentru licitare
        print("Trimit licitatie mea: {}".format(self.my_bid))
        bid_message = bytearray("licitez", encoding="utf-8") # corpul contine
doar mesajul "licitez"
        bid_headers = [ # antetul mesajului contine identitatea ofertantului
si, respectiv, oferta sa
            ("amount", self.my_bid.to_bytes(2, byteorder='big')),
            ("identity", bytes("Bidder {}".format(self.my_id),
encoding="utf-8"))
        ]

        # se trimite licitatie sub forma de mesaj catre Kafka
        self.bid_producer.send(topic=self.bids_topic, value=bid_message,
headers=bid_headers)

        # exista o sansa din 2 ca oferta sa fie trimisa de 2 ori pentru a
simula duplicatele
        if randint(0, 1) == 1:
            self.bid_producer.send(topic=self.bids_topic, value=bid_message,
headers=bid_headers)

```

```

        self.bid_producer.flush()
        self.bid_producer.close()

    def get_winner(self):
        # se asteapta raspunsul licitatiei
        print("Astept rezultatul licitatiei...")
        result = next(self.result_consumer)

        # se verifica identitatea castigatorului
        for header in result.headers:
            if header[0] == "identity":
                identity = str(header[1], encoding="utf-8")

        if identity == "Bidder {}".format(self.my_id):
            print("[{}] Am castigat!!!".format(self.my_id))
        else:
            print("[{}] Am pierdut...".format(self.my_id))

        self.result_consumer.close()

    def run(self):
        self.bid()
        self.get_winner()

if __name__ == '__main__':
    bidder = Bidder(
        bids_topic="topic_oferte",
        result_topic="topic_rezultat"
    )
    bidder.run()

```

BiddingProcessor

```

from kafka import KafkaConsumer, KafkaProducer

class BiddingProcessor:
    def __init__(self, processed_bids_topic, result_topic):
        super().__init__()
        self.processed_bids_topic = processed_bids_topic
        self.result_topic = result_topic

        # consumatorul pentru ofertele procesate
        self.processed_bids_consumer = KafkaConsumer(
            self.processed_bids_topic,
            auto_offset_reset="earliest", # mesajele se preiau de la cel mai
            # vechi la cel mai recent

```

```

        consumer_timeout_ms=3000
    )

    # producatorul pentru trimiterea rezultatului licitatiei
    self.result_producer = KafkaProducer()

def get_processed_bids(self):
    # se preiau toate ofertele procesate din topicul processed_bids_topic
    print("Astept ofertele procesate de MessageProcessor...")

    # ofertele se stocheaza sub forma de perechi <PRET_LICITAT,
MESAJ_OFERTA>
    bids = dict()
    no_bids_available = True

    while no_bids_available:
        for msg in self.processed_bids_consumer:
            for header in msg.headers:
                if header[0] == "amount":
                    bid_amount = int.from_bytes(header[1], 'big')
                    bids[bid_amount] = msg

            # daca inca nu exista oferte, se asteapta in continuare
            if len(bids) != 0:
                no_bids_available = False

    self.processed_bids_consumer.close()
    self.decide_auction_winner(bids)

def decide_auction_winner(self, bids):
    print("Procesez ofertele...")

    if len(bids) == 0:
        print("Nu exista nicio oferta de procesat.")
        return

    # sortare dupa oferte, descrescator
    sorted_bids = sorted(bids.keys(), reverse=True)

    # castigatorul este ofertantul care a oferit pretul cel mai mare
    winner = bids[sorted_bids[0]]

    for header in winner.headers:
        if header[0] == "identity":
            winner_identity = str(header[1], encoding="utf-8")

    print("Castigatorul este:")

```

```

        print("\t{} - pret licitat: {}".format(winner_identity,
sorted_bids[0]))

        # se trimite rezultatul licitatiei pentru ca entitatile Bidder sa il
preia din topicul corespunzator
        self.result_producer.send(topic=self.result_topic, value=winner.value,
headers=winner.headers)
        self.result_producer.flush()
        self.result_producer.close()

    def run(self):
        self.get_processed_bids()

if __name__ == '__main__':
    bidding_processor = BiddingProcessor(
        processed_bids_topic="topic_oferte_procesate",
        result_topic="topic_rezultat"
    )
    bidding_processor.run()

```

MessageProcessor

```

from datetime import datetime
from kafka import KafkaConsumer, KafkaProducer

class MessageProcessor:
    def __init__(self, bids_topic, notify_message_processor_topic,
processed_bids_topic):
        super().__init__()
        self.bids_topic = bids_topic
        self.notify_message_processor_topic = notify_message_processor_topic
        self.processed_bids_topic = processed_bids_topic

        # consumatorul notificarii de la Auctioneer cum ca s-a terminat
licitatia
        self.notify_message_processor_consumer = KafkaConsumer(
            self.notify_message_processor_topic,
            auto_offset_reset="earliest" # mesajele se preiau de la cel mai
vechi la cel mai recent
        )

        # consumatorul pentru ofertele de la licitatie
        self.bids_consumer = KafkaConsumer(
            self.bids_topic,
            auto_offset_reset="earliest",

```

```

        consumer_timeout_ms=1000
    )

    # producatorul pentru mesajele procesate
    self.processed_bids_producer = KafkaProducer()

    # ofertele se pun in dictionar, sub forma de perechi
    <IDENTITATE_OFERTANT, MESAJ_OFERTA>
    self.bids = dict()

    def get_and_process_messages(self):
        # se asteapta notificarea de la Auctioneer pentru incheierea licitatiei
        print("Astept notificare de la toate entitatile Auctioneer pentru
incheierea licitatiei...")
        auction_end_message = next(self.notify_message_processor_consumer)

        # a ajuns prima notificare, se asteapta si celelalte notificari timp de
maxim 15 secunde
        self.notify_message_processor_consumer.config["consumer_timeout_ms"] =
15_000
        for auction_end_message in self.notify_message_processor_consumer:
            pass
        self.notify_message_processor_consumer.close()

        if str(auction_end_message.value, encoding="utf-8") == "incheiat":
            # se preiau toate ofertele din topicul bids_topic si se proceseaza
            print("Licitatie incheiata. Procesez mesajele cu oferte...")

            for msg in self.bids_consumer:
                for header in msg.headers:
                    if header[0] == "identity":
                        identity = str(header[1], encoding="utf-8")

                        # eliminare duplicate
                        if identity not in self.bids.keys():
                            self.bids[identity] = msg

            self.bids_consumer.close()

            # sortare dupa timestamp
            sorted_bids = sorted(self.bids.values(), key=lambda bid:
bid.timestamp)

            self.finish_processing(sorted_bids)

    def finish_processing(self, sorted_bids):
        print("Procesarea s-a incheiat! Trimit urmatoarele oferte:")
        for bid in sorted_bids:
            for header in bid.headers:

```

```

        if header[0] == "identity":
            identity = str(header[1], encoding="utf-8")
        elif header[0] == "amount":
            bid_amount = int.from_bytes(header[1], 'big')
        print("{} {} a licitat
    {}.format(datetime.fromtimestamp(bid.timestamp / 1000), identity,
bid_amount))

    # se stocheaza mesajele ordonate dupa timestamp si fara duplicate
    intr-un topic separat
    self.processed_bids_producer.send(topic=self.processed_bids_topic,
value=bid.value, headers=bid.headers)

    self.processed_bids_producer.flush()
    self.processed_bids_producer.close()

    def run(self):
        self.get_and_process_messages()

if __name__ == '__main__':
    message_processor = MessageProcessor(
        bids_topic="topic_oferte",
        notify_message_processor_topic="topic_notificare_procesor_mesaje",
        processed_bids_topic="topic_oferte_procesate"
    )
    message_processor.run()

```

PrepareAuction

```

import pika

if __name__ == '__main__':
    connection =
pika.BlockingConnection(pika.ConnectionParameters('localhost'))
    channel = connection.channel()

    channel.queue_declare(queue='queue1')

    print("Gata! Microserviciile participante la licitatie pot fi pornite.")

```